

Experiment 'p_14_1_selection_sort_desc_array' Results

June 5, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 2

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_14_1_selection_sort_desc_array

Natural language statement: Write a method that implements the selection sort algorithm to sort an array of integers in descending order.

Method signature: p_14_1_selection_sort_desc_array(inputs: array<int>) returns (sorted: array<int>)

Ensures

- forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] >= sorted[j]
- multiset(inputs[..]) == multiset(sorted[..])

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that implements the selection sort algorithm to sort an array of integers
→ in descending order.

The signature should be:

p_14_1_selection_sort_desc_array(inputs: array<int>) returns (sorted: array<int>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] >= sorted[j], ensures
→ multiset(inputs[..]) == multiset(sorted[..])

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnKEgSd3CxSXgDVpfegFfcpsluHRm

Model: gpt-5.5-2026-04-23

Created at: 1780648230

Finish reason: stop

Usage: 330 tokens in, and 10451 tokens out

```
{
  sorted := new int[inputs.Length];
  var n := inputs.Length;

  var c := 0;
  while c < n
    invariant 0 <= c <= n
    invariant n == inputs.Length
    invariant sorted.Length == n
    invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
  {
    sorted[c] := inputs[c];
    c := c + 1;
  }
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := 0;
```

```

while i < n
  invariant 0 <= i <= n
  invariant n == inputs.Length
  invariant sorted.Length == n
  invariant multiset(sorted[..]) == multiset(inputs[..])
  invariant forall p, q :: 0 <= p < q < i ==> sorted[p] >= sorted[q]
  invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] >= sorted[q]
{
  var maxIdx := i;
  var j := i + 1;
  while j < n
    invariant 0 <= i < n
    invariant i + 1 <= j <= n
    invariant i <= maxIdx < j
    invariant n == inputs.Length
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] >= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] >= sorted[q]
    invariant forall k :: i <= k < j ==> sorted[maxIdx] >= sorted[k]
  {
    if sorted[j] > sorted[maxIdx] {
      assert forall k | i <= k < j ensures sorted[j] >= sorted[k]
      {
        assert sorted[maxIdx] >= sorted[k];
      }
      maxIdx := j;
    } else {
      assert sorted[maxIdx] >= sorted[j];
    }
    assert forall k | i <= k < j + 1 ensures sorted[maxIdx] >= sorted[k]
    {
      if k < j {
        assert sorted[maxIdx] >= sorted[k];
      } else {
        assert k == j;
      }
    }
  }
  j := j + 1;
}

assert j == n;
ghost var a := sorted[..];
assert |a| == n;
assert multiset(a) == multiset(inputs[..]);
assert forall k | i <= k < n ensures a[maxIdx] >= a[k]
{
  assert sorted[maxIdx] >= sorted[k];
}
assert forall p, q | 0 <= p < q < i ensures a[p] >= a[q]
{

```

```

    assert sorted[p] >= sorted[q];
}
assert forall p, q | 0 <= p < i <= q < n ensures a[p] >= a[q]
{
    assert sorted[p] >= sorted[q];
}

if maxIdx != i {
    sorted[i], sorted[maxIdx] := sorted[maxIdx], sorted[i];
    assert sorted[..] == a[i := a[maxIdx]][maxIdx := a[i]];
    assert multiset(sorted[..]) == multiset(a);
} else {
    assert sorted[..] == a;
}
assert multiset(sorted[..]) == multiset(a);
assert multiset(sorted[..]) == multiset(inputs[..]);

assert sorted[i] == a[maxIdx];
assert sorted[maxIdx] == a[i];
assert forall k | 0 <= k < n && k != i && k != maxIdx ensures sorted[k] == a[k]
{
}

var ni := i + 1;
assert ni <= n;

assert forall p, q | 0 <= p < q < ni ensures sorted[p] >= sorted[q]
{
    if q < i {
        assert a[p] >= a[q];
        assert p != i && p != maxIdx;
        assert q != i && q != maxIdx;
        assert sorted[p] == a[p];
        assert sorted[q] == a[q];
    } else {
        assert q == i;
        assert p < i;
        assert i <= maxIdx < n;
        assert a[p] >= a[maxIdx];
        assert p != i && p != maxIdx;
        assert sorted[p] == a[p];
        assert sorted[q] == a[maxIdx];
    }
}

assert forall p, q | 0 <= p < ni <= q < n ensures sorted[p] >= sorted[q]
{
    if p < i {
        assert p != i && p != maxIdx;
        assert sorted[p] == a[p];
        if q == maxIdx {

```

```

        assert sorted[q] == a[i];
        assert a[p] >= a[i];
    } else {
        assert q != i;
        assert sorted[q] == a[q];
        assert a[p] >= a[q];
    }
} else {
    assert p == i;
    assert sorted[p] == a[maxIdx];
    if q == maxIdx {
        assert sorted[q] == a[i];
        assert a[maxIdx] >= a[i];
    } else {
        assert q != i;
        assert sorted[q] == a[q];
        assert a[maxIdx] >= a[q];
    }
}
}

i := ni;
}

assert i == n;
assert sorted.Length == n;
assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] >= sorted[q];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_14_1_selection_sort_desc_array/program.dfy(49,15): Error: invalid
  ↳ AssertStmt

```

```

|
49 | maxIdx := j;
| ^

```

1 parse errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the

- ↳ Dafny body, including the curly braces that surround it. Do not show the signature
- ↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnKIEEy4UWBj4svMDoZKRMMyTY1VIK

Model: gpt-5.5-2026-04-23

Created at: 1780648450

Finish reason: stop

Usage: 1815 tokens in, and 2187 tokens out

```
{
  sorted := new int[inputs.Length];
  var n := inputs.Length;

  var c := 0;
  while c < n
    invariant 0 <= c <= n
    invariant n == inputs.Length
    invariant sorted.Length == n
    invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
  {
    sorted[c] := inputs[c];
    c := c + 1;
  }
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := 0;
  while i < n
    invariant 0 <= i <= n
    invariant n == inputs.Length
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] >= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] >= sorted[q]
  {
    var maxIdx := i;
    var j := i + 1;
    while j < n
      invariant 0 <= i < n
      invariant i + 1 <= j <= n
      invariant i <= maxIdx < j
      invariant n == inputs.Length
      invariant sorted.Length == n
      invariant multiset(sorted[..]) == multiset(inputs[..])
      invariant forall p, q :: 0 <= p < q < i ==> sorted[p] >= sorted[q]
      invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] >= sorted[q]
      invariant forall k :: i <= k < j ==> sorted[maxIdx] >= sorted[k]
    {
      if sorted[j] > sorted[maxIdx] {
        forall k | i <= k < j
          ensures sorted[j] >= sorted[k]
        {
          assert sorted[maxIdx] >= sorted[k];
        }
        maxIdx := j;
      }
    }
  }
}
```

```

    } else {
      assert sorted[maxIdx] >= sorted[j];
    }
    forall k | i <= k < j + 1
      ensures sorted[maxIdx] >= sorted[k]
    {
      if k < j {
        assert sorted[maxIdx] >= sorted[k];
      } else {
        assert k == j;
      }
    }
    j := j + 1;
  }

  assert j == n;
  ghost var a := sorted[..];
  assert |a| == n;
  assert multiset(a) == multiset(inputs[..]);
  forall k | i <= k < n
    ensures a[maxIdx] >= a[k]
  {
    assert sorted[maxIdx] >= sorted[k];
  }
  forall p, q | 0 <= p < q < i
    ensures a[p] >= a[q]
  {
    assert sorted[p] >= sorted[q];
  }
  forall p, q | 0 <= p < i <= q < n
    ensures a[p] >= a[q]
  {
    assert sorted[p] >= sorted[q];
  }

  if maxIdx != i {
    sorted[i], sorted[maxIdx] := sorted[maxIdx], sorted[i];
    assert sorted[..] == a[i := a[maxIdx]][maxIdx := a[i]];
    assert multiset(sorted[..]) == multiset(a);
  } else {
    assert sorted[..] == a;
  }
  assert multiset(sorted[..]) == multiset(a);
  assert multiset(sorted[..]) == multiset(inputs[..]);

  assert sorted[i] == a[maxIdx];
  assert sorted[maxIdx] == a[i];
  forall k | 0 <= k < n && k != i && k != maxIdx
    ensures sorted[k] == a[k]
  {
  }

```

```

var ni := i + 1;
assert ni <= n;

forall p, q | 0 <= p < q < ni
  ensures sorted[p] >= sorted[q]
{
  if q < i {
    assert a[p] >= a[q];
    assert p != i && p != maxIdx;
    assert q != i && q != maxIdx;
    assert sorted[p] == a[p];
    assert sorted[q] == a[q];
  } else {
    assert q == i;
    assert p < i;
    assert i <= maxIdx < n;
    assert a[p] >= a[maxIdx];
    assert p != i && p != maxIdx;
    assert sorted[p] == a[p];
    assert sorted[q] == a[maxIdx];
  }
}

forall p, q | 0 <= p < ni <= q < n
  ensures sorted[p] >= sorted[q]
{
  if p < i {
    assert p != i && p != maxIdx;
    assert sorted[p] == a[p];
    if q == maxIdx {
      assert sorted[q] == a[i];
      assert a[p] >= a[i];
    } else {
      assert q != i;
      assert sorted[q] == a[q];
      assert a[p] >= a[q];
    }
  } else {
    assert p == i;
    assert sorted[p] == a[maxIdx];
    if q == maxIdx {
      assert sorted[q] == a[i];
      assert a[maxIdx] >= a[i];
    } else {
      assert q != i;
      assert sorted[q] == a[q];
      assert a[maxIdx] >= a[q];
    }
  }
}

```



```

    i := ni;
  }

  assert i == n;
  assert sorted.Length == n;
  assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] >= sorted[q];
  assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Final Program

```

method p_14_1_selection_sort_desc_array(inputs: array<int>) returns (sorted: array<int>)
  ensures forall i, j :: 0 <= i < j < sorted.Length ==> sorted[i] >= sorted[j]
  ensures multiset(inputs[..]) == multiset(sorted[..])
{
  sorted := new int[inputs.Length];
  var n := inputs.Length;

  var c := 0;
  while c < n
    invariant 0 <= c <= n
    invariant n == inputs.Length
    invariant sorted.Length == n
    invariant forall k :: 0 <= k < c ==> sorted[k] == inputs[k]
  {
    sorted[c] := inputs[c];
    c := c + 1;
  }
  assert forall k :: 0 <= k < n ==> sorted[k] == inputs[k];
  assert sorted[..] == inputs[..];
  assert multiset(sorted[..]) == multiset(inputs[..]);

  var i := 0;
  while i < n
    invariant 0 <= i <= n
    invariant n == inputs.Length
    invariant sorted.Length == n
    invariant multiset(sorted[..]) == multiset(inputs[..])
    invariant forall p, q :: 0 <= p < q < i ==> sorted[p] >= sorted[q]
    invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] >= sorted[q]
  {
    var maxIdx := i;
    var j := i + 1;
    while j < n
      invariant 0 <= i < n
      invariant i + 1 <= j <= n
      invariant i <= maxIdx < j
      invariant n == inputs.Length
      invariant sorted.Length == n

```

```

invariant multiset(sorted[..]) == multiset(inputs[..])
invariant forall p, q :: 0 <= p < q < i ==> sorted[p] >= sorted[q]
invariant forall p, q :: 0 <= p < i <= q < n ==> sorted[p] >= sorted[q]
invariant forall k :: i <= k < j ==> sorted[maxIdx] >= sorted[k]
{
  if sorted[j] > sorted[maxIdx] {
    forall k | i <= k < j
      ensures sorted[j] >= sorted[k]
    {
      assert sorted[maxIdx] >= sorted[k];
    }
    maxIdx := j;
  } else {
    assert sorted[maxIdx] >= sorted[j];
  }
  forall k | i <= k < j + 1
    ensures sorted[maxIdx] >= sorted[k]
  {
    if k < j {
      assert sorted[maxIdx] >= sorted[k];
    } else {
      assert k == j;
    }
  }
  j := j + 1;
}

assert j == n;
ghost var a := sorted[..];
assert |a| == n;
assert multiset(a) == multiset(inputs[..]);
forall k | i <= k < n
  ensures a[maxIdx] >= a[k]
{
  assert sorted[maxIdx] >= sorted[k];
}
forall p, q | 0 <= p < q < i
  ensures a[p] >= a[q]
{
  assert sorted[p] >= sorted[q];
}
forall p, q | 0 <= p < i <= q < n
  ensures a[p] >= a[q]
{
  assert sorted[p] >= sorted[q];
}

if maxIdx != i {
  sorted[i], sorted[maxIdx] := sorted[maxIdx], sorted[i];
  assert sorted[..] == a[i := a[maxIdx]][maxIdx := a[i]];
  assert multiset(sorted[..]) == multiset(a);
}

```

```

} else {
  assert sorted[..] == a;
}
assert multiset(sorted[..]) == multiset(a);
assert multiset(sorted[..]) == multiset(inputs[..]);

assert sorted[i] == a[maxIdx];
assert sorted[maxIdx] == a[i];
forall k | 0 <= k < n && k != i && k != maxIdx
  ensures sorted[k] == a[k]
{
}

var ni := i + 1;
assert ni <= n;

forall p, q | 0 <= p < q < ni
  ensures sorted[p] >= sorted[q]
{
  if q < i {
    assert a[p] >= a[q];
    assert p != i && p != maxIdx;
    assert q != i && q != maxIdx;
    assert sorted[p] == a[p];
    assert sorted[q] == a[q];
  } else {
    assert q == i;
    assert p < i;
    assert i <= maxIdx < n;
    assert a[p] >= a[maxIdx];
    assert p != i && p != maxIdx;
    assert sorted[p] == a[p];
    assert sorted[q] == a[maxIdx];
  }
}

forall p, q | 0 <= p < ni <= q < n
  ensures sorted[p] >= sorted[q]
{
  if p < i {
    assert p != i && p != maxIdx;
    assert sorted[p] == a[p];
    if q == maxIdx {
      assert sorted[q] == a[i];
      assert a[p] >= a[i];
    } else {
      assert q != i;
      assert sorted[q] == a[q];
      assert a[p] >= a[q];
    }
  }
} else {

```

```

    assert p == i;
    assert sorted[p] == a[maxIdx];
    if q == maxIdx {
        assert sorted[q] == a[i];
        assert a[maxIdx] >= a[i];
    } else {
        assert q != i;
        assert sorted[q] == a[q];
        assert a[maxIdx] >= a[q];
    }
}
}

i := ni;
}

assert i == n;
assert sorted.Length == n;
assert forall p, q :: 0 <= p < q < sorted.Length ==> sorted[p] >= sorted[q];
assert multiset(inputs[..]) == multiset(sorted[..]);
}

```

Total Token Usage

Input tokens: 2145

Output tokens: 12638

Reasoning tokens: 9731

Sum of 'total tokens': 14783

Experiment Timings

Overall Experiment started at 1780648229505, ended at 1780648498570, lasting 269065ms (269.07 seconds)

Iteration #1 started at 1780648229507, ended at 1780648444883, lasting 215376ms (215.38 seconds)

Iteration #2 started at 1780648444884, ended at 1780648498563, lasting 53679ms (53.68 seconds)